PDF Processing Guide

IMPORTANT: Do NOT Use the Read Tool for PDFs

The Read tool cannot properly extract tabular data from PDFs. It will only show you a limited preview of the first page's text content, missing most of the data.

For PDF files, especially multi-page PDFs with tables:
- **DO**: Use `pdfplumber` as shown in this skill
- **DO NOT**: Use the Read tool to view PDF contents

If you need to extract data from a PDF, write Python code using pdfplumber. This is the only reliable way to get complete table data from all pages.

Overview

Extract text and tables from PDF documents using Python libraries.

Table Extraction with pdfplumber

pdfplumber is the recommended library for extracting tables from PDFs.

Basic Table Extraction

```python
import pdfplumber

with pdfplumber.open("document.pdf") as pdf:
for page in pdf.pages:
tables = page.extract_tables()
for table in tables:
print(table) # List of lists
```

Convert to pandas DataFrame

```python
import pdfplumber
import pandas as pd

with pdfplumber.open("document.pdf") as pdf:
page = pdf.pages[0] # First page
tables = page.extract_tables()

if tables:
# First row is usually headers
table = tables[0]
df = pd.DataFrame(table[1:], columns=table[0])
print(df)
```

Extract All Tables from Multi-Page PDF

```python
import pdfplumber
import pandas as pd

all_tables = []

with pdfplumber.open("document.pdf") as pdf:
for page in pdf.pages:
tables = page.extract_tables()
for table in tables:
if table and len(table) > 1: # Has data
df = pd.DataFrame(table[1:], columns=table[0])
all_tables.append(df)

# Combine if same structure
if all_tables:
combined = pd.concat(all_tables, ignore_index=True)
```

```

## Text Extraction

### Full Page Text

```python
import pdfplumber

with pdfplumber.open("document.pdf") as pdf:
text = ""
for page in pdf.pages:
text += page.extract_text() + "\n"
```

Text with Layout Preserved

```python
with pdfplumber.open("document.pdf") as pdf:
page = pdf.pages[0]
text = page.extract_text(layout=True)
```

Common Patterns

Extract Multiple Tables and Create Mappings

When a PDF contains multiple related tables (possibly spanning multiple pages), extract from ALL pages and build lookup dictionaries:

```python
import pdfplumber
import pandas as pd

category_map = {} # CategoryID -> CategoryName
product_map = {} # ProductID -> (ProductName, CategoryID)

with pdfplumber.open("catalog.pdf") as pdf:
for page_num, page in enumerate(pdf.pages):
tables = page.extract_tables()

for table in tables:
if not table or len(table) < 2:
continue

# Check first row to identify table type
header = [str(cell).strip() if cell else '' for cell in table[0]]

# Determine if first row is header or data (continuation page)
if 'CategoryID' in header and 'CategoryName' in header:
# Categories table with header
for row in table[1:]:
if row and len(row) >= 2 and row[0]:
```
```

cat_id = int(row[0].strip())
cat_name = row[1].strip()
category_map[cat_id] = cat_name

elif 'ProductID' in header and 'ProductName' in header:
# Products table with header
for row in table[1:]:
if row and len(row) >= 3 and row[0]:
try:
prod_id = int(row[0].strip())
prod_name = row[1].strip()
cat_id = int(row[2].strip())
product_map[prod_id] = (prod_name, cat_id)
except (ValueError, AttributeError):
continue
else:
# Continuation page (no header) - check if it's product data
for row in table:
if row and len(row) >= 3 and row[0]:
try:
prod_id = int(row[0].strip())
prod_name = row[1].strip()
cat_id = int(row[2].strip())
product_map[prod_id] = (prod_name, cat_id)
except (ValueError, AttributeError):
continue

# Build final mapping: ProductID -> CategoryName
product_to_category_name = {
pid: category_map[cat_id]
for pid, (name, cat_id) in product_map.items()
}
# {1: 'Beverages', 2: 'Beverages', 3: 'Condiments', ...}
```

**Important:** Always iterate over ALL pages (`for page in pdf.pages`) - tables often span multiple pages, and continuation pages may not have headers.

### Clean Extracted Table Data

```python
import pdfplumber
import pandas as pd

with pdfplumber.open("document.pdf") as pdf:
table = pdf.pages[0].extract_tables()[0]
df = pd.DataFrame(table[1:], columns=table[0])

# Clean whitespace
df = df.apply(lambda x: x.str.strip() if x.dtype == "object" else x)

```
# Remove empty rows
df = df.dropna(how='all')

# Rename columns if needed
df.columns = df.columns.str.strip()
```

Handle Missing Headers

```python
# If table has no header row
table = pdf.pages[0].extract_tables()[0]
df = pd.DataFrame(table)
df.columns = ['Col1', 'Col2', 'Col3', 'Col4'] # Assign manually
```

Quick Reference

Task	Code
Open PDF	`pdfplumber.open("file.pdf")`
Get pages	`pdf.pages`
Extract tables	`page.extract_tables()`
Extract text	`page.extract_text()`
Table to DataFrame	`pd.DataFrame(table[1:], columns=table[0])`

Troubleshooting

- **Empty table**: Try `page.extract_tables(table_settings={...})` with custom settings
- **Merged cells**: Tables with merged cells may extract incorrectly
- **Scanned PDFs**: Use OCR (pytesseract) for image-based PDFs
- **No tables found**: Check if content is actually a table or styled text